

Assignment 1: Part 1

1. A brief outline of the application

My application - MyRightHand (link to repository in Appendix 1) - is a digital phonebook that uses the command-line interface (CLI) to display information. Users can navigate menus by typing letters corresponding to the on-screen options. Users are able to perform the following actions within the application:

- Add a new contact (demonstrating insertion);
- View existing contacts (in alphabetical order by name, demonstrating sorting);
- Edit a contact (choosing to edit just the name, just the number, or both the name and number, demonstrating insertion);
- Delete a contact (demonstrating deletion);
- Search for a contact (demonstrating searching).

After searching and finding a contact, users can also choose to do the following:

- Edit the found contact (demonstrating insertion);
- Delete the found contact (demonstrating deletion).

The application executes by pressing 'run' and the above features are accessed by typing letters corresponding to the on-screen prompts (see

flowcharts (i) (ii) (iii) to see these prompts on pages 3, 4, 5, and 12, respectively).

2. Design of data structure and algorithms

The following numbered flowcharts demonstrate the data flow and data structures that the application utilises, following object-oriented programming. The first three flowcharts demonstrate how the three main menus function. These menus are instantiated from the Menu class, from which the 'show_menu' method is inherited. The methods called from the menus belong to the my_phonebook object that is instantiated from the Phonebook Class.

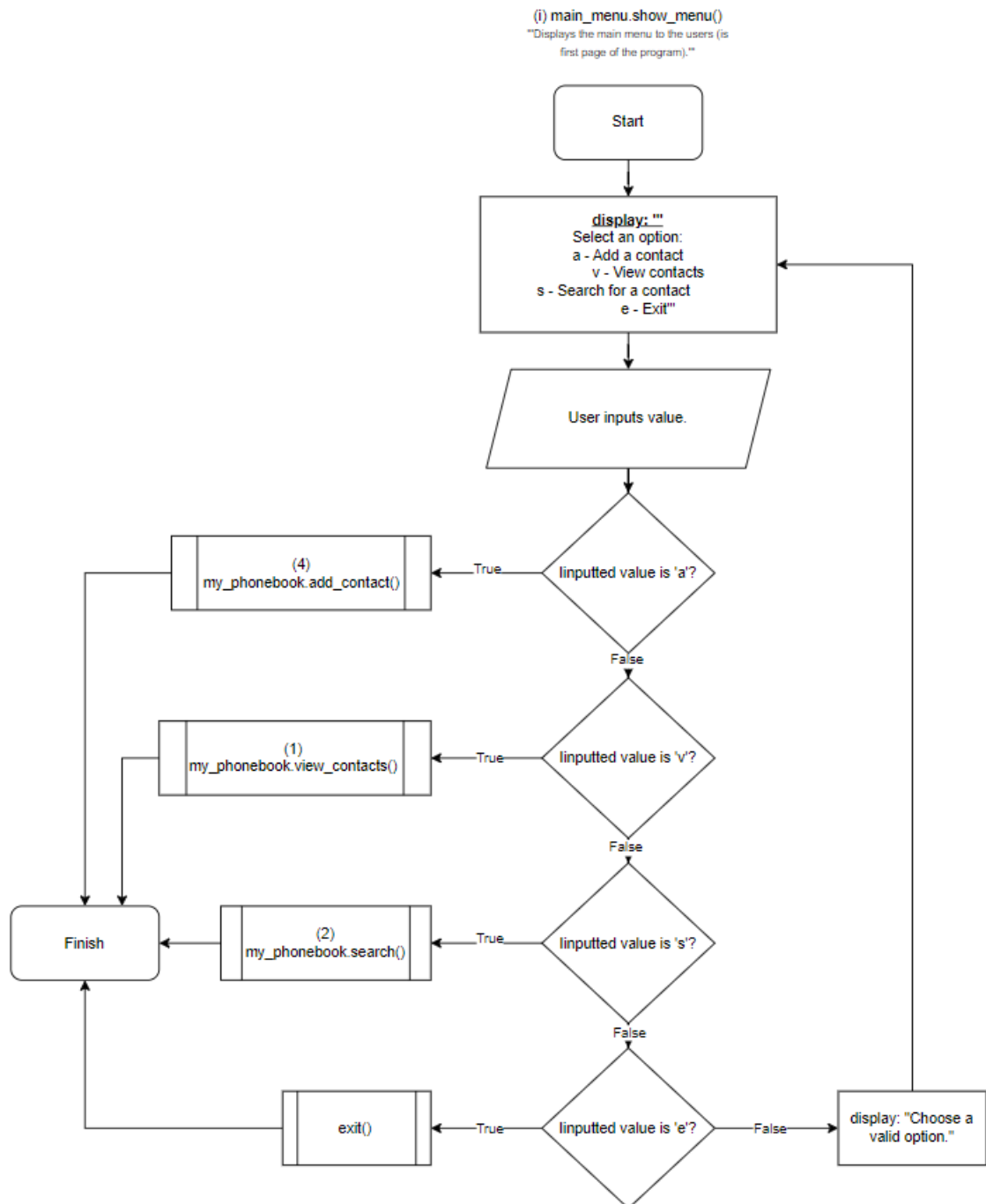
Key insertion, deletion, sorting, and searching algorithms are demonstrated within the following methods:

Insertion: (7) my_phonebook.insert_contact(self, name, number) [page 11]

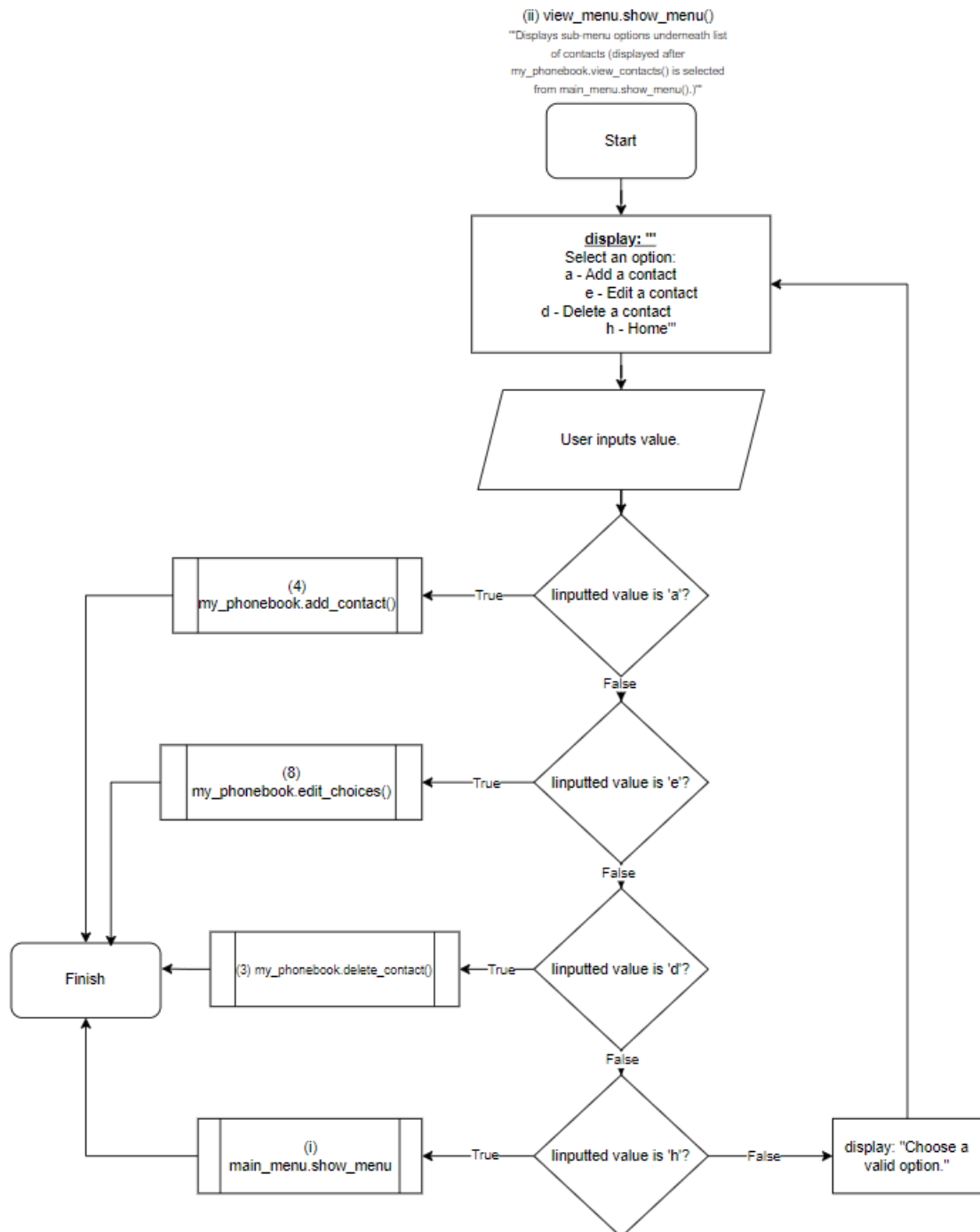
Deletion: (3) my_phonebook.delete_contact(self) [page 10]

Sorting: (1a) my_phonebook.bubble_sort(self, contact_list) [page 7]

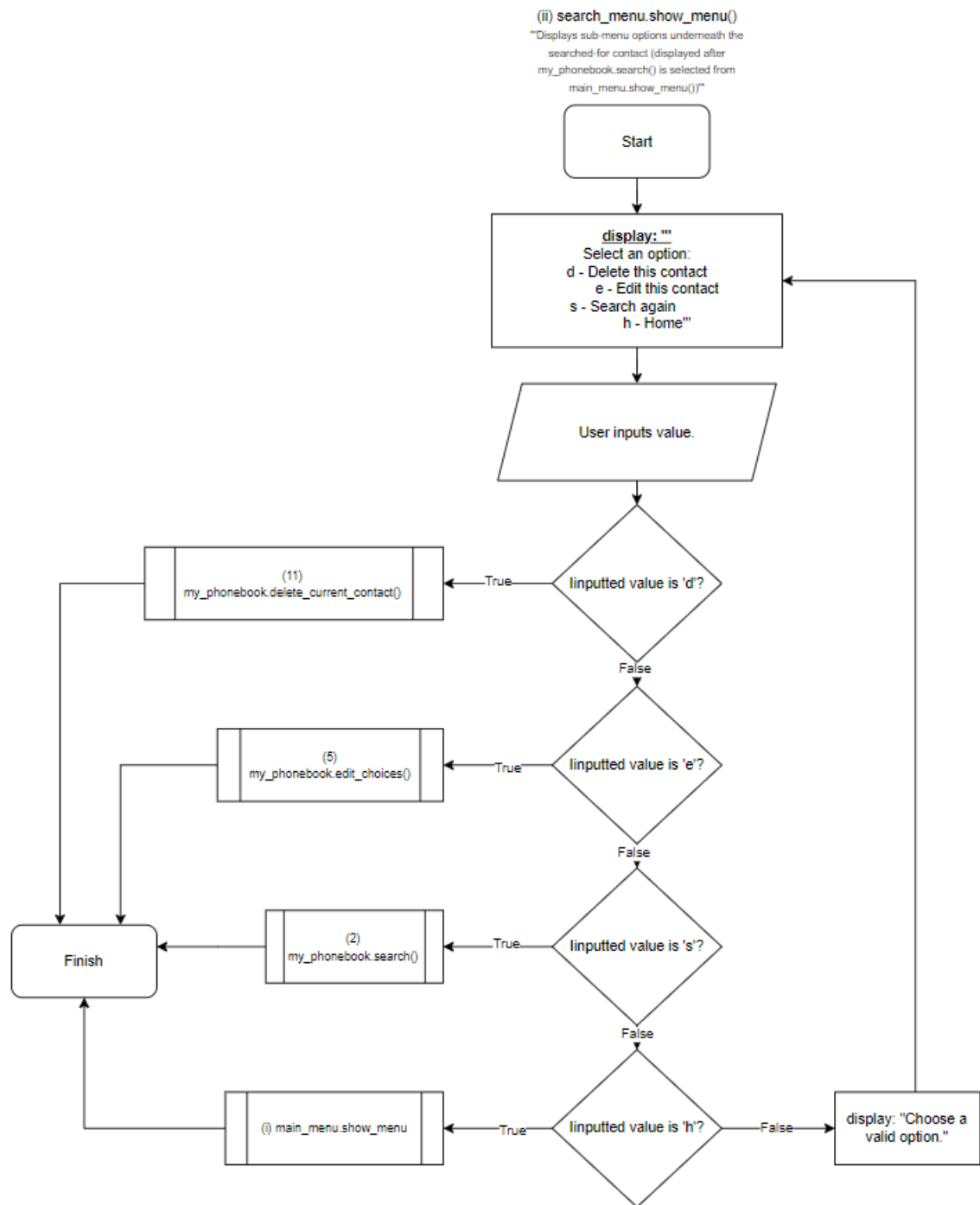
Searching: (2a) my_phonebook.binary_search(self) [page 9]



Idea for algorithm inspired by HyperionDev (2018).



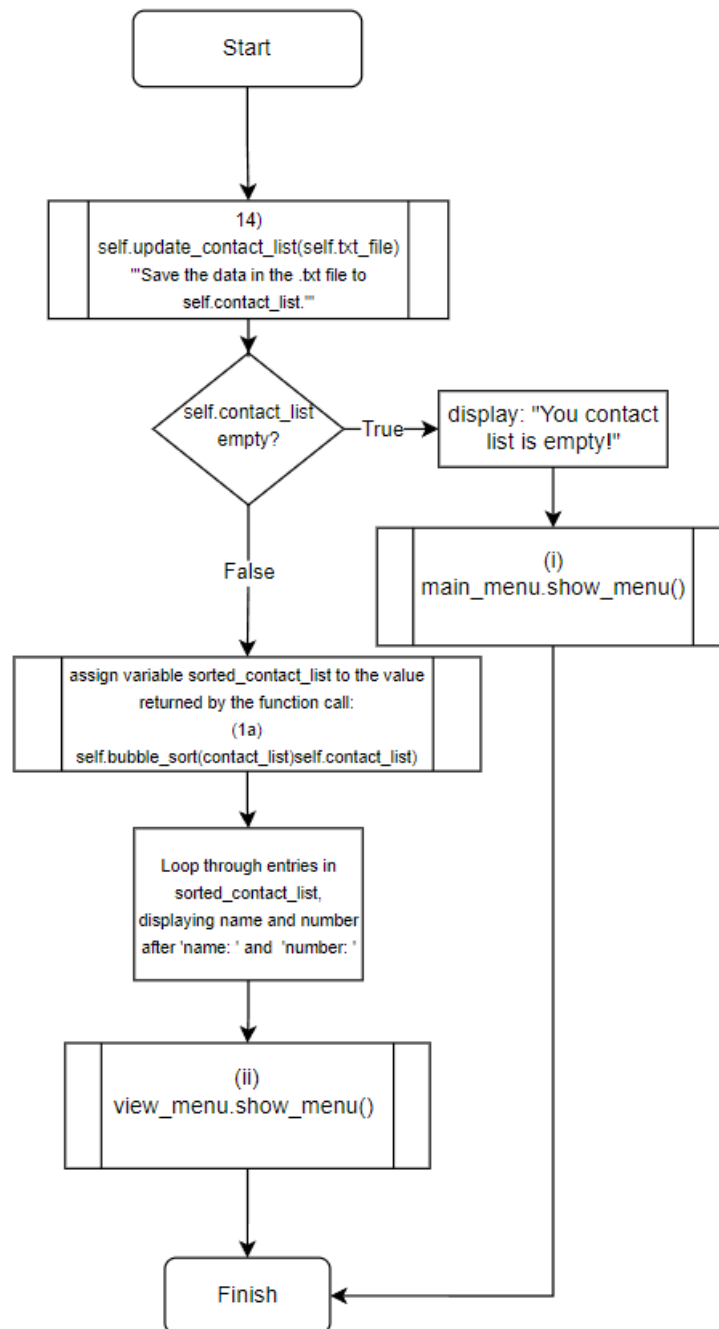
Idea for algorithm inspired by HyperionDev (2018).



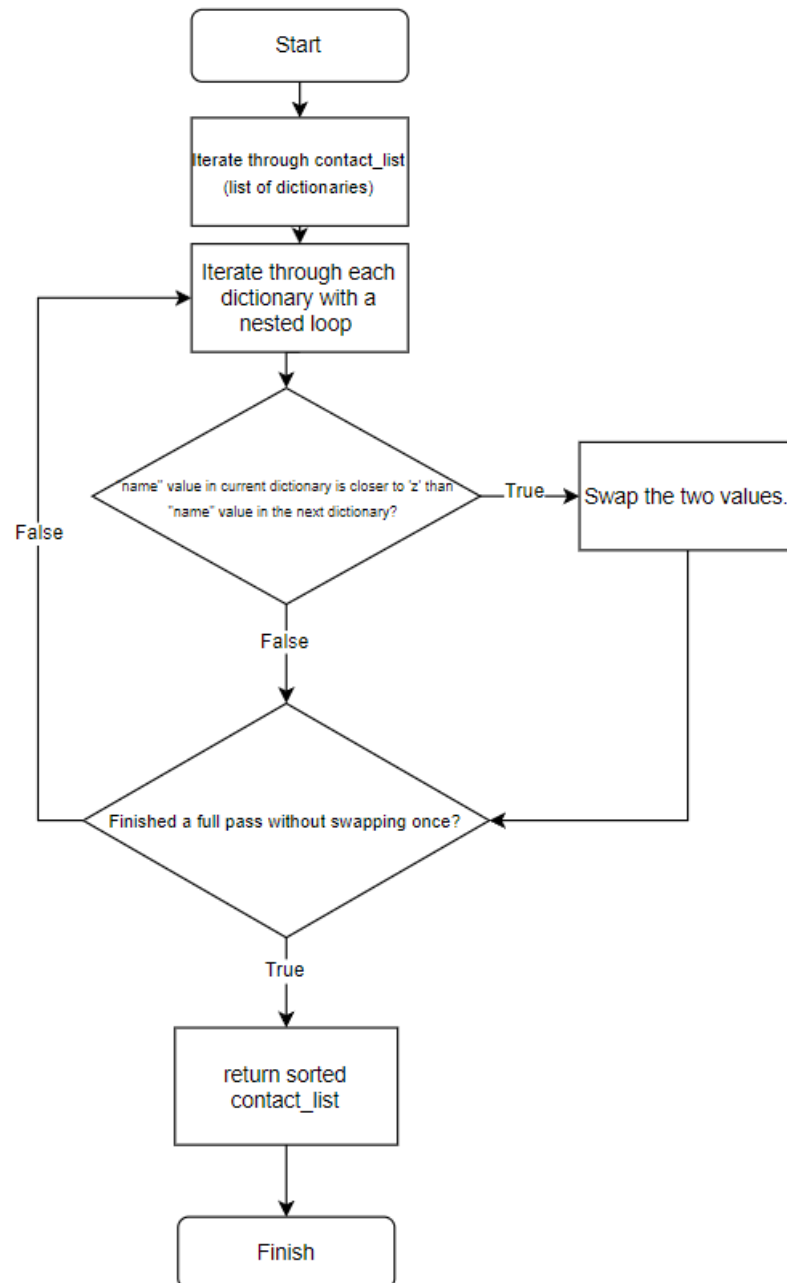
Idea for algorithm inspired by HyperionDev (2018).

(1) my_phonebook.view_contacts()

""Displays the contacts in alphabetical order
by name.""

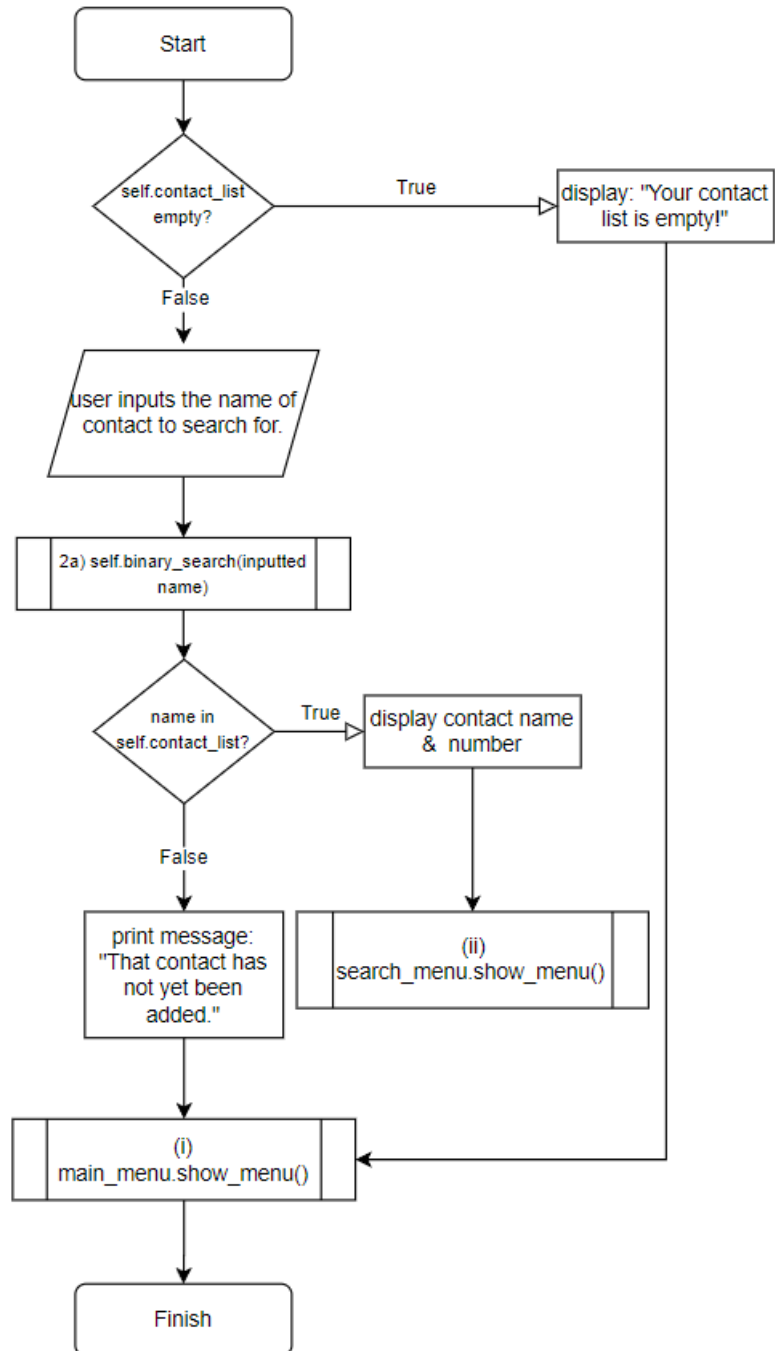


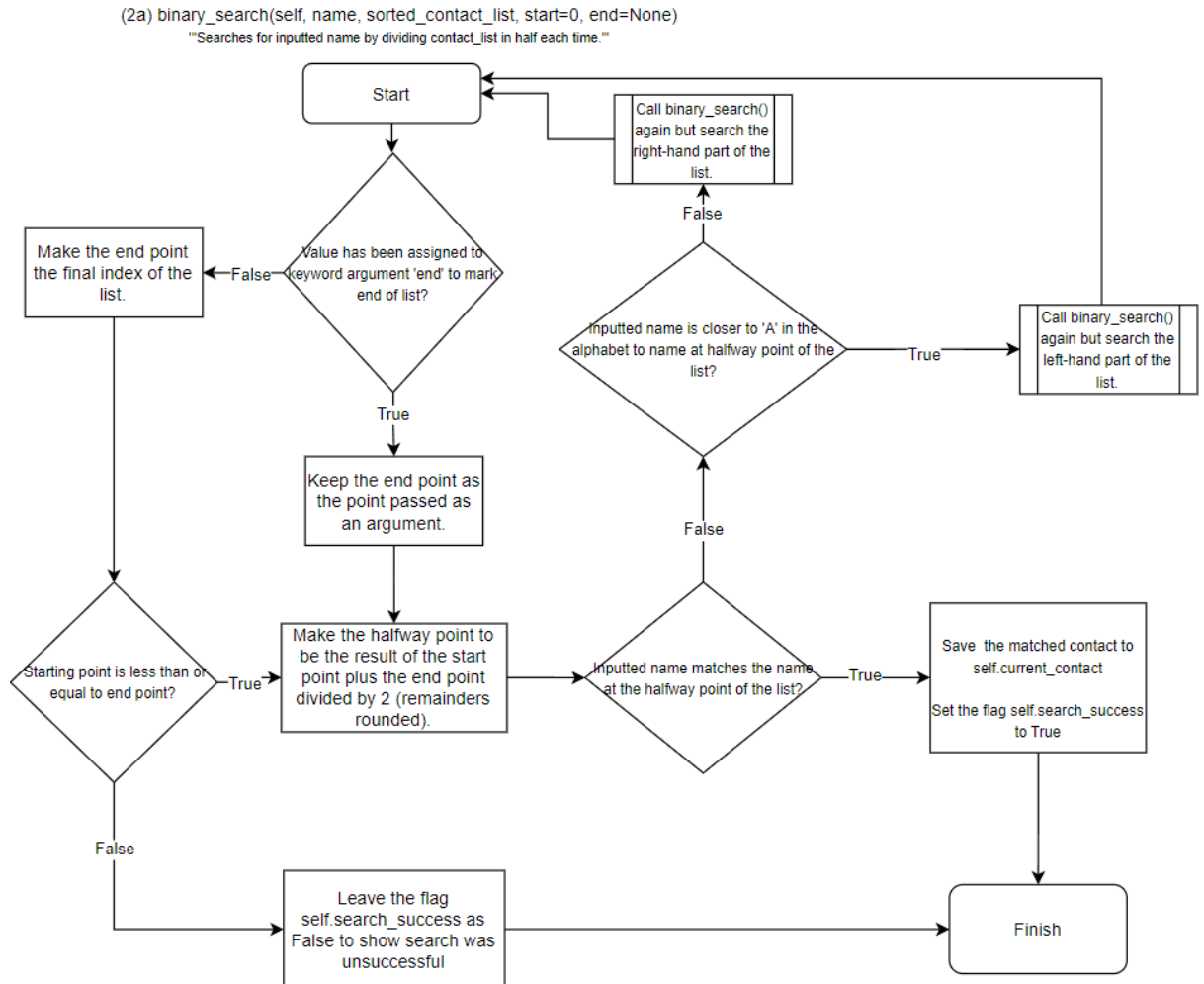
(1a) my_phonebook.bubble_sort(contact_list)
"""Sorts the contact list alphabetically by name."""



(2) my_phonebook.search()

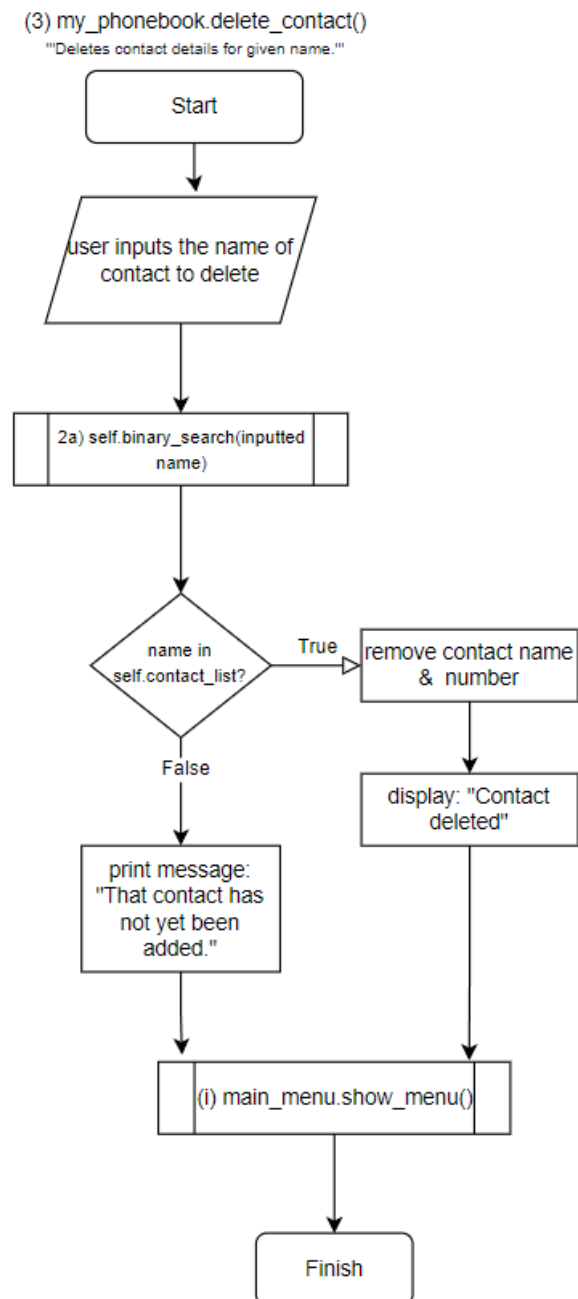
""Searches for a contact (by name) when 'search' is discretely selected and then displays name and number.""



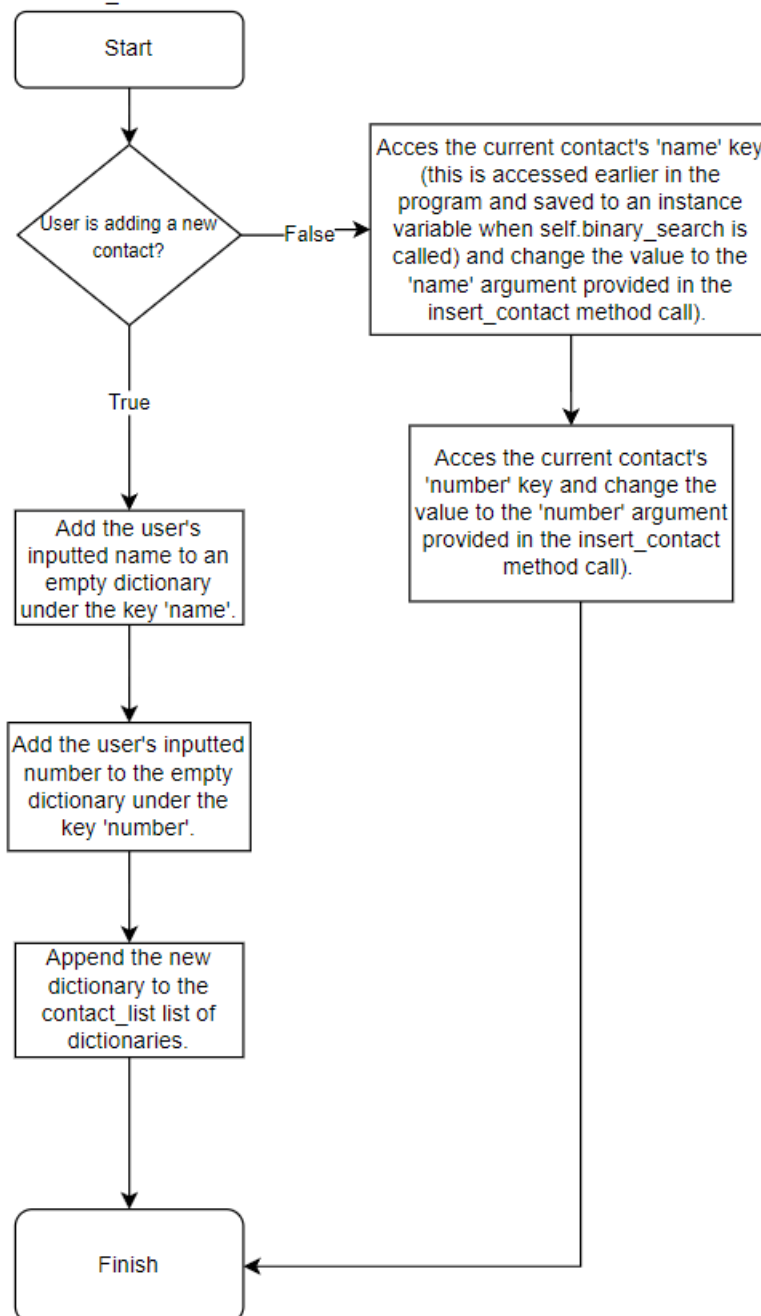


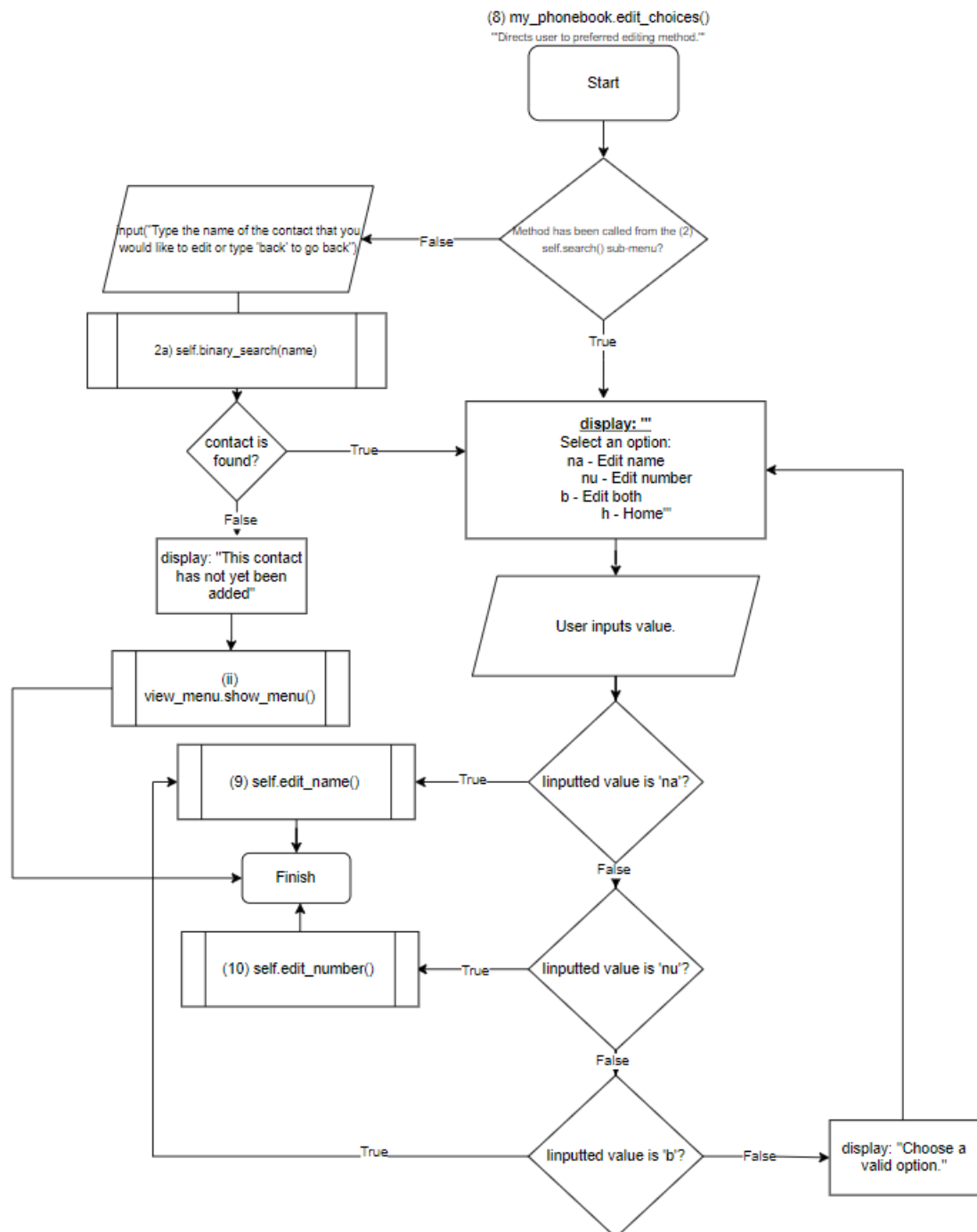
When prompted by the author with a question about a draft binary search algorithm, ChatGPT (OpenAI ChatGPT, 2024) responded with a suggested fix

that this algorithm is based on. A copy of the prompt and response is in Appendix 1.

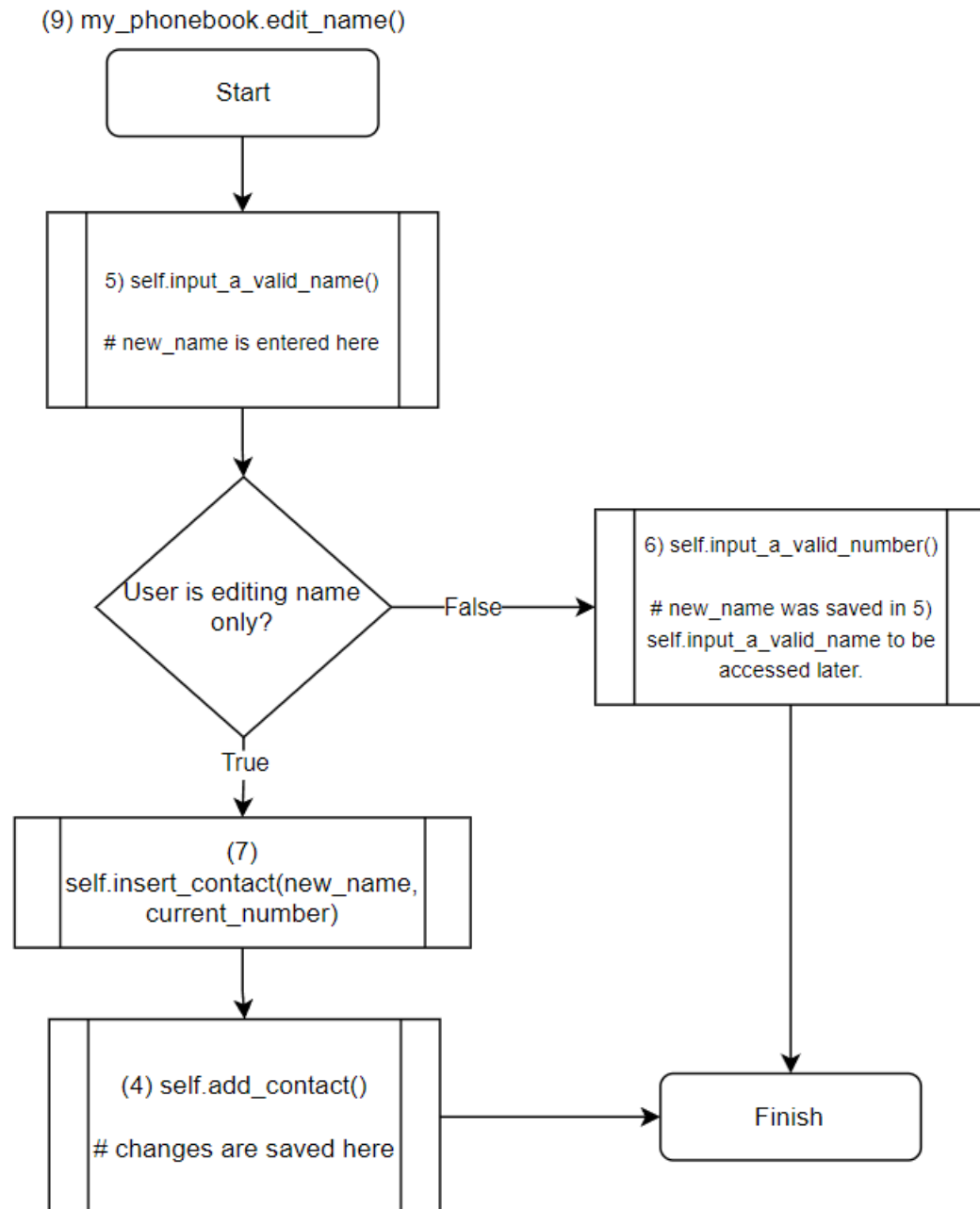


```
(7) my_phonebook.insert_contact(self, name,  
                                number)  
    """Inserts an edited contact or new contact  
    into the self.contact_list list of dictionaries."""
```

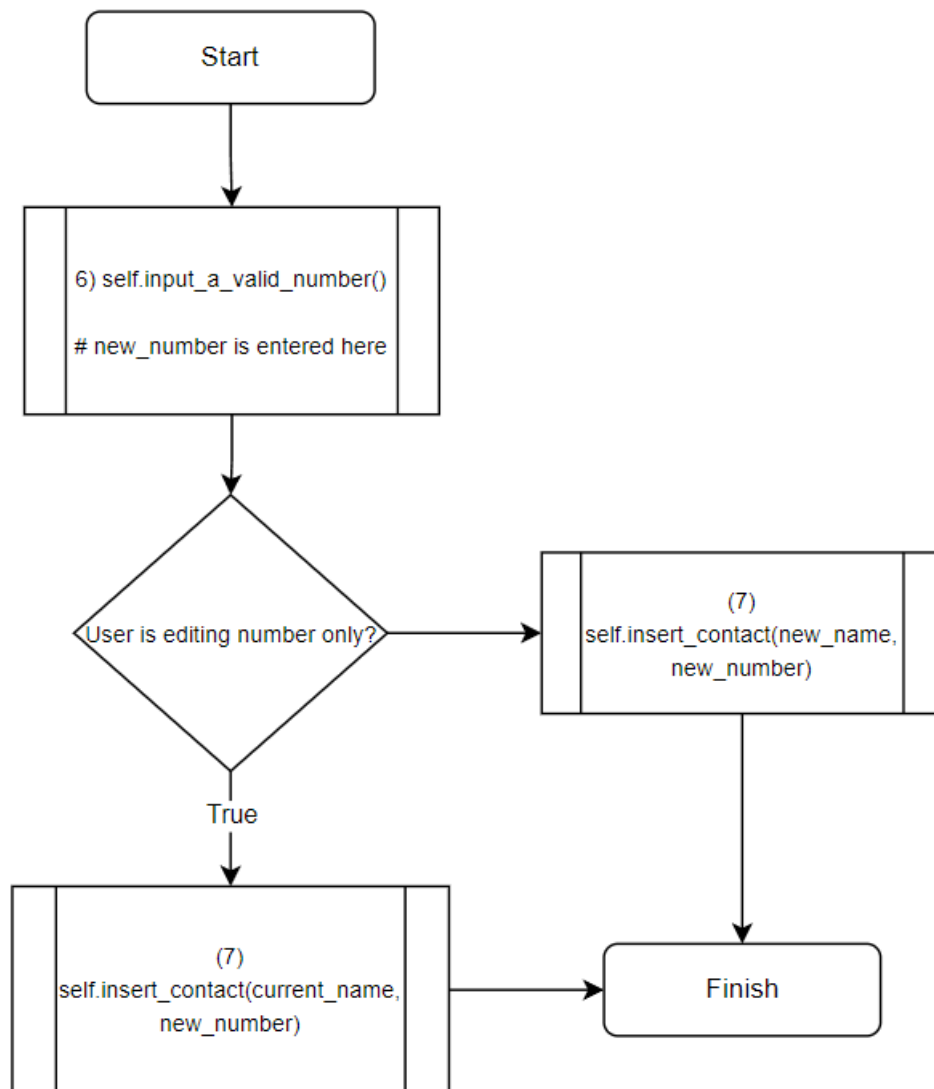




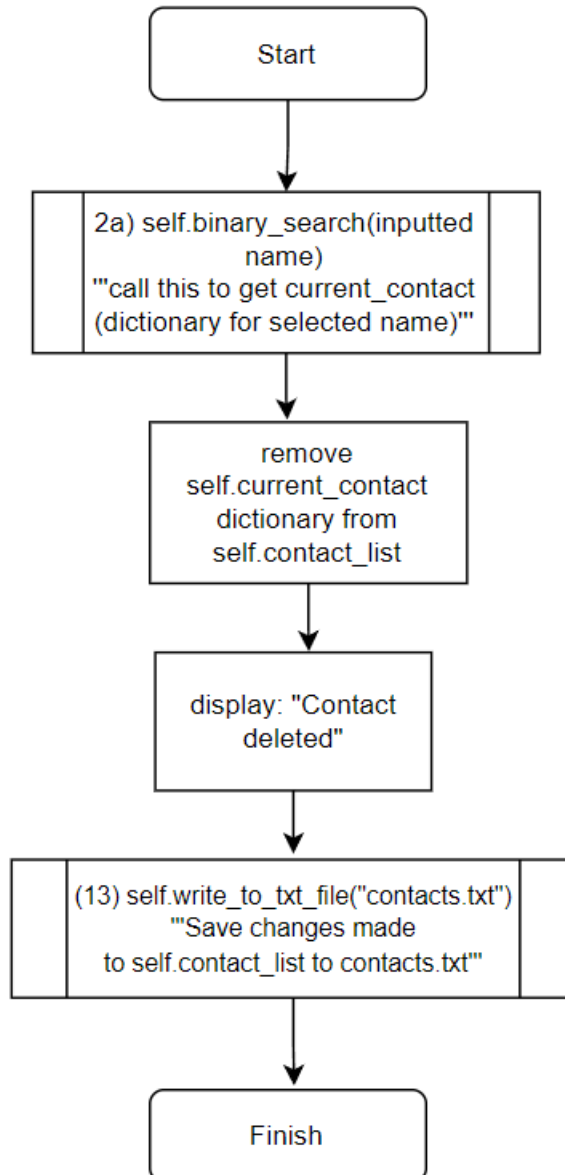
Idea for algorithm inspired by HyperionDev (2018).



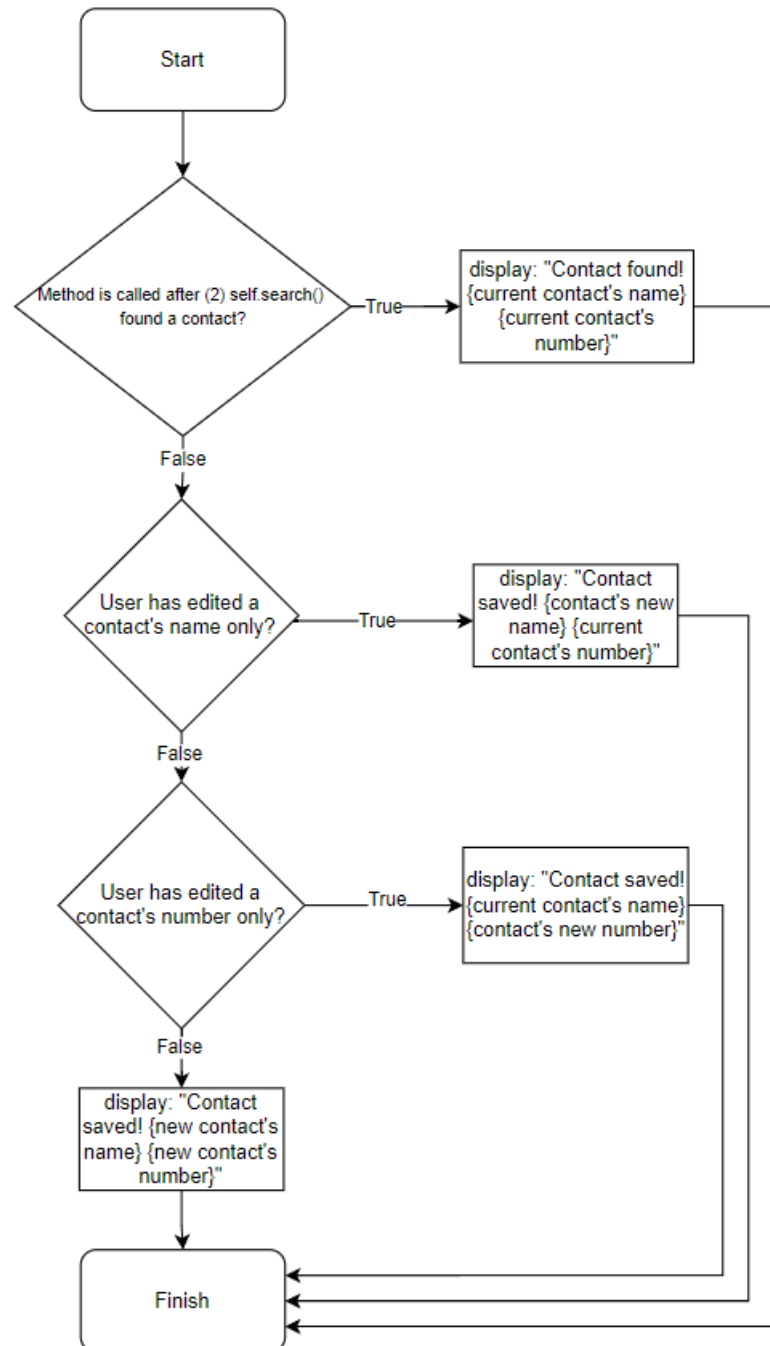
(10) my_phonebook.edit_number()



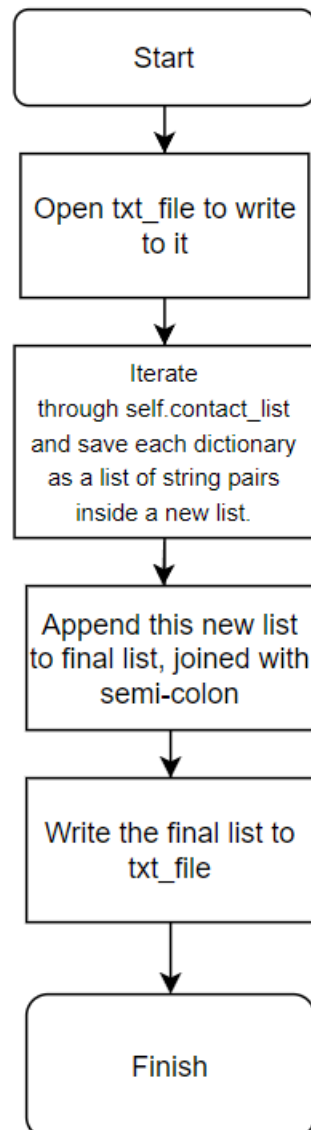
(11) my_phonebook.delete_current_contact()
"""Deletes the current contact when selected
from the discrete search sub-menu."""



(12) my_phonebook.display_success_of_search_or_change_or_addition()
""Displays a confirmation of searched-for, added or edited contact.""

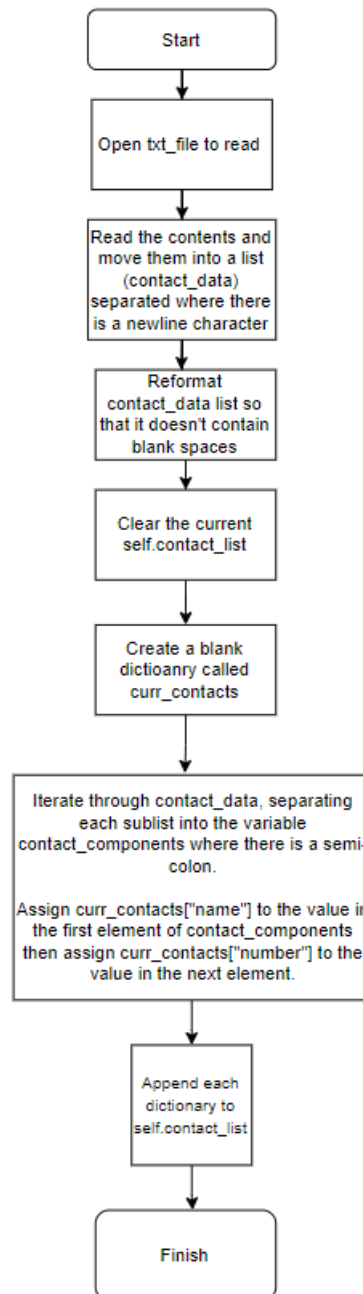


(13) my_phonebook.write_to_txt_file(txt_file)
 """Writes changes made to the contacts in
 self.contact_list to a .txt file."""



Idea for algorithm inspired by HyperionDev (2018).

```
(14) my_phonebook.update_contact_list(txt_file)
    """Saves the updated .txt file to contact_list."""
```



Idea for algorithm inspired by HyperionDev (2018).

3. Test plan

STEP ID	DESCRIPTION	STEPS	INPUTS	EXPECTED RESULTS
1	<u>Insertion</u> Testing the ability to 'add a contact' by entering a name and number to add the new contact to the 'contacts.txt' file.	1) Run program; 2) Type 'a' to add a contact; 3) Enter 'test' then press Enter when prompted for a name; 4) Enter 1234 then press Enter when prompted; 5) Repeat the above with following: name: Bob, number: 2345; name: Lisa, number: 3456; name: Bobby, number: 4567. 5) Type 'v' at the next menu to view contacts and check to see if the contact added is listed there; 6) Confirm that contact has	test' (when prompted for name); 1234 (when prompted for number)	The following is visible after adding contact and typing 'v' to view contacts on subsequent menu: 'name: test number: 1234' The following is visible in contacts.txt file: 'test;1234'

		saved to 'contacts.txt' by exiting the program and viewing the file.		
2	<u>Sorting</u> Testing the ability of the 'view_contacts' function to sort the contacts in alphabetical order (by name).	1) Run program; 2) Type 'v' to view contacts.	Type 'v' (when prompted at first menu).	The following is visible (in this order): 'name: Bob number: 2345 name: Bobby number: 4567 name: Lisa number: 3456 name: test number: 1234'

3	<u>Deletion</u> Testing the ability to delete a valid name that is known to be saved in 'contacts.txt'.	1) Run program; 2) Type 'd' to delete a contact; 3) Type 'test' then press Enter when prompted for the name of a contact to delete; 4) Press 'v' on the next menu to view contacts and check results.	new name' (when prompted for a name to delete)	The following is visible once the enter key is pressed: ** Contact deleted ** The following is visible once 'v' is pressed on next menu: 'name: Bob number: 2345 name: Bobby number: 4567 name: Lisa number: 3456 The following is visible within contacts.txt file: 'Bob;2345 Lisa;3456 Bobby;4567'
4	<u>Searching</u> Testing the ability to search for a name that is known to be saved in 'contacts.txt'.	1) Run program; 2) Type 's' to search contacts; 3) Type 'boBby' then press Enter when prompted for a name to search for.	boBby' (when prompted for a name to search for)	The following is visible once the name has been entered: ** Contact found ** name: Bobby number: 4567'

Conclusion

The key data structure within this application is a list of dictionaries (saved in the instance variable `my_phonebook.contact_list`). Each dictionary contains a record of the user's name and number under a 'name' key and 'number' key respectively. The data from this dictionary is written to the 'contacts.txt' file – with each contact saved on a separate line, with name and number separated with a semicolon - at the end of a change brought about from insertion or deletion.

References

HyperionDev (2018) *task_manager.py* (Version 2018) [computer program]. Available at

https://github.com/tedge85/task_manager/commit/b6cae61aa8668160ab21a7f68071c7a8d0e3482a (Downloaded: 18 December 2023).

OpenAIChatGPT (2024) ChatGPT response to Todd Edge [code]. Available at

<https://copilot.microsoft.com/sl/kXemM4MQPII> (Accessed: 20 June 2024).

Appendix 1

1) my-right-hand repository:

<https://github.com/tedge85/my-right-hand>

2) OpenAI ChatGPT (2024) prompt and response.

Prompt by author:

‘Why does this binary search keep repeating?’

```
dict_list = [{"name": "Todd", "number": 1}, {"name": "Codd", "number": 1}, {"name": "Fodd", "number": 1}, {"name": "Lodd", "number": 1}, {"name": "Kodd", "number": 1}]
```

```
def bubble_sort(self, contact_list):
```

```
    """Sorts the contact list alphabetically by name."""
```

```
    swapped = False
```

```
    n = len(contact_list)
```

```
    for i in range(n):
```

```
        # Nested loop to iterate through each dictionary, indexed as 'j'
```

```
        for j in range(0, n-i-1):
```

```

        # Variables for name values in dictionaries next to each other in
contact_list.

        name1 = contact_list[j]["name"]
        name2 = contact_list[j+1]["name"]

        # Swap contact order if name value closer to 'a' in alphabet.
        if name1 > name2:

            contact_list[j], contact_list[j+1] = contact_list[j+1], contact_list[j]
            swapped = True

        # Exit outer loop and stop comparing if single swap not made (i.e. list already
in order).
        if not swapped:
            break

    return contact_list

def binary_search(name, contact_list):

    # Iterate through list to match, divide each time by indexing, recursion
    halfway = round(len(contact_list)/2 -1)
    end = len(contact_list)
    print(halfway)
    print(contact_list)

    if halfway >= 0: if contact_list[halfway]["name"].lower() == name.lower():
        print("search success")
    elif contact_list[halfway]["name"].lower() > name.lower():

```

```

        new_list = contact_list[:halfway]
        binary_search(name, new_list)
    elif contact_list[halfway]["name"].lower() < name.lower():
        new_list = contact_list[halfway:end]
        binary_search(name, new_list)
    else: print("search fail")

```

```
sorted_list = bubble_sort(dict_list) binary_search("Todd", sorted_list)
```

Response from OpenAI ChatGPT:

The issue with the binary search function is that it does not correctly update the range of the list to search through after each recursive call. Specifically, when the search continues in the upper half of the list, it should not include the halfway index again, as this can cause an infinite loop if the halfway index is where the search term is found or if there's no change in the index due to rounding.

Here's how you can fix it:

Python

```

def binary_search(name, contact_list, start=0, end=None):
    if end is None:
        end = len(contact_list) - 1

    if start <= end:
        halfway = (start + end) // 2 # Use integer division to avoid floating-point indices
        print(halfway)

```

```
print(contact_list[start:end+1]) # Print the current range being searched
```

```
if contact_list[halfway]["name"].lower() == name.lower():
```

```
    print("search success")
```

```
    return halfway # Return the index if found
```

```
elif contact_list[halfway]["name"].lower() > name.lower():
```

```
    return binary_search(name, contact_list, start, halfway - 1) # Search in the
left half
```

```
else:
```

```
    return binary_search(name, contact_list, halfway + 1, end) # Search in the
right half
```

```
else:
```

```
    print("search fail")
```

```
    return None # Return None if not found
```

AI-generated code. Review and use carefully. [More info on FAQ.](#)

In this revised version of `binary_search`, I've added `start` and `end` parameters to keep track of the current range being searched. This ensures that with each recursive call, the function searches a smaller section of the list until it finds the target or determines that it's not present. Also, I've used integer division (`//`) for calculating half-way to ensure we get an integer index. Give this updated function a try! 🔗🔍

